

Hotel Booking Platform MVP Architecture (Booking.com / Airbnb / Expedia Style)

1. Recommended Tech Stack

Frontend

- React / Next.js
- TypeScript
- Tailwind CSS
- Redux Toolkit or Zustand

Backend

- Node.js
- NestJS or Express.js
- GraphQL or REST API
- Socket.IO for real-time updates

Database

- PostgreSQL
- Redis
- Elasticsearch or Meilisearch for search

Infrastructure

- AWS
- Cloudflare
- Docker
- Kubernetes (later stage)

Payments

- Stripe
 - PayPal
 - Flutterwave
 - Payoutstack
-

2. Core Business Model

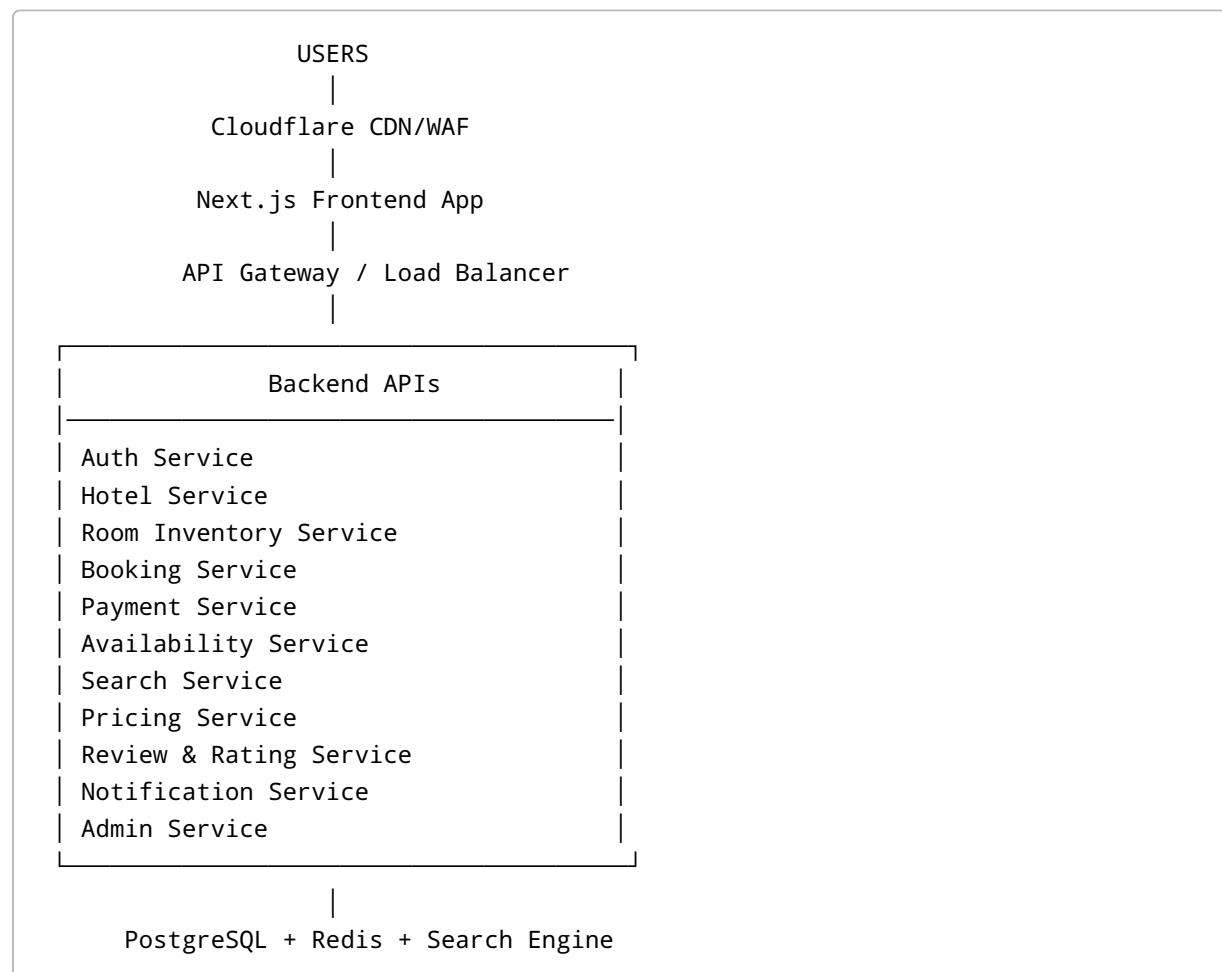
The platform connects:

1. Guests
2. Hotels/Property Owners
3. Admins

The platform earns revenue through:

- Booking commissions
- Featured hotel listings
- Subscription plans
- Ads/promoted properties
- Service fees

3. High-Level Architecture



4. Frontend Pages

Public Pages

Page	Purpose
Home	Featured hotels & destinations
Search Results	Hotel listings
Hotel Details	Rooms, pricing, amenities
Room Details	Individual room details
Checkout	Booking/payment flow
Login/Register	Authentication
Promotions	Deals & offers

Guest Dashboard

Page	Purpose
My Bookings	Booking history
Wishlist	Saved hotels
Wallet	Credits/refunds
Reviews	Submitted reviews
Notifications	Booking updates
Profile	Guest profile

Hotel Owner Dashboard

Page	Purpose
Overview	Analytics

Page	Purpose
Properties	Manage hotels
Rooms	Manage room inventory
Pricing	Dynamic pricing
Bookings	Reservation management
Payouts	Earnings
Reviews	Guest feedback

Admin Dashboard

Page	Purpose
Users	Manage users
Hotels	Approve/manage hotels
Bookings	Reservation oversight
Payments	Transactions
Reports	Revenue analytics
Fraud Monitoring	Suspicious activity
CMS	Homepage banners/content

5. Backend Modules

A. Auth Module

Responsibilities

- Registration/login
- JWT authentication
- Password reset
- Email verification
- 2FA support

APIs

```
POST /auth/register  
POST /auth/login  
POST /auth/logout  
POST /auth/reset-password  
GET /auth/me
```

B. User Module

Responsibilities

- Profiles
- Preferences
- Saved hotels
- Travel history

APIs

```
GET /users/profile  
PUT /users/profile  
GET /users/wishlist  
POST /users/wishlist
```

C. Hotel Module

Responsibilities

- Hotel onboarding
- Property management
- Amenities
- Policies
- Hotel images

APIs

```
POST /hotels  
PUT /hotels/:id  
GET /hotels/:id  
GET /hotels  
DELETE /hotels/:id
```

D. Room Inventory Module

Responsibilities

- Room listings
- Room availability
- Occupancy management
- Seasonal pricing

APIs

```
POST /rooms  
PUT /rooms/:id  
GET /rooms/:id  
GET /rooms/hotel/:hotelId
```

E. Search Module

Responsibilities

- Destination search
- Date filtering
- Price filtering
- Amenities filtering
- Availability filtering

APIs

```
GET /search?destination=lagos  
GET /search/suggestions
```

F. Booking Module

Responsibilities

- Reservation creation
- Availability locking
- Booking modifications
- Cancellations

APIs

```
POST /bookings
GET /bookings/:id
GET /bookings/me
PUT /bookings/:id/cancel
```

G. Payment Module

Responsibilities

- Card payments
- Refunds
- Hotel payouts
- Multi-currency support

APIs

```
POST /payments/initialize
POST /payments/webhook
POST /payments/refund
```

H. Pricing Module

Responsibilities

- Dynamic pricing
- Discounts
- Seasonal pricing
- Coupon management

APIs

```
POST /pricing/rules
GET /pricing/hotel/:id
```

I. Review & Rating Module

Responsibilities

- Hotel reviews
- Room reviews
- Rating aggregation
- Moderation

APIs

POST /reviews

GET /reviews/hotel/:id

6. Core Database Schema

users

Field	Type
id	UUID
email	VARCHAR
password_hash	TEXT
role	ENUM
created_at	TIMESTAMP

hotels

Field	Type
id	UUID
owner_id	UUID
name	VARCHAR
description	TEXT
address	TEXT
city	VARCHAR

Field	Type
country	VARCHAR
status	ENUM

hotel_images

Field	Type
id	UUID
hotel_id	UUID
image_url	TEXT

rooms

Field	Type
id	UUID
hotel_id	UUID
name	VARCHAR
description	TEXT
price_per_night	DECIMAL
capacity	INTEGER

room_inventory

Field	Type
id	UUID
room_id	UUID
available_rooms	INTEGER
date	DATE

bookings

Field	Type
id	UUID
user_id	UUID
hotel_id	UUID
room_id	UUID
check_in	DATE
check_out	DATE
total_amount	DECIMAL
status	ENUM

payments

Field	Type
id	UUID
booking_id	UUID
provider	VARCHAR
amount	DECIMAL
status	ENUM

reviews

Field	Type
id	UUID
hotel_id	UUID
user_id	UUID
rating	INTEGER
comment	TEXT

7. Booking Flow

1. User searches destination
2. User selects hotel
3. User chooses room
4. Availability checked
5. Booking initiated
6. Payment completed
7. Booking confirmed
8. Notifications sent
9. Hotel receives reservation

8. Search Architecture

Use Elasticsearch or Meilisearch for:

- Fast hotel search
- Autocomplete
- Geo-based search
- Filters
- Ranking and recommendations

Indexes:

- Hotels
- Rooms
- Cities
- Amenities

9. Real-Time Features

Use Socket.IO or WebSockets for:

- Booking confirmations
 - Availability updates
 - Price changes
 - Chat support
 - Notifications
-

10. Security Architecture

Required

- JWT authentication
- HTTPS only
- Rate limiting
- Input validation
- Secure payments
- File upload validation
- WAF protection

Fraud Prevention

- Fake booking detection
 - Chargeback monitoring
 - Suspicious login detection
 - Hotel verification
-

11. MVP Features Only

Include

✓ Hotel listings ✓ Room inventory ✓ Booking engine ✓ Payments ✓ Search filters ✓ Hotel owner dashboard ✓ Reviews ✓ Admin dashboard

Skip Initially

✗ Flight booking ✗ AI recommendations ✗ Loyalty program ✗ Native mobile apps ✗ Dynamic package deals ✗ Multi-language support

12. Deployment Plan

Phase 1 — MVP Hosting

Frontend

- Vercel

Backend

- Railway
- Render

Database

- Supabase PostgreSQL
- AWS RDS

Object Storage

- AWS S3
 - Cloudflare R2
-

Phase 2 — Scale

Move to:

- AWS ECS
 - Kubernetes
 - Dedicated Redis cluster
 - CDN optimization
 - Read replicas
 - Queue systems
-

13. Suggested Folder Structure

Frontend

```
/apps/web
  /components
  /pages
  /hooks
  /store
  /services
  /types
```

Backend

```
/apps/api
  /modules
    /auth
    /users
    /hotels
    /rooms
    /bookings
    /payments
    /search
  /workers
  /common
```

14. Revenue Model

Hotel marketplace revenue sources:

1. Booking commissions
2. Featured hotel listings
3. Ads/promoted hotels
4. Subscription plans
5. Service fees

Example:

- Booking amount: \$200
- Platform commission: 15%
- Hotel receives: \$170

15. Recommended MVP Timeline

Stage	Time
UI/UX Design	1–2 weeks
Backend APIs	3–5 weeks
Booking Engine	2 weeks
Payment Integration	1 week
Search Integration	1 week

Stage	Time
Testing	1–2 weeks
Deployment	2–3 days

16. Scaling Considerations

As traffic grows:

- Separate services into microservices
 - Add queue systems
 - Introduce caching layers
 - Optimize search indexes
 - Use CDN image optimization
 - Add recommendation engines
-

17. Best Next Step

Fastest launch strategy:

1. Start with one city or region
2. Launch web-first
3. Verify hotel owners manually
4. Focus on smooth booking flow
5. Add mobile apps later
6. Expand inventory gradually

A strong MVP can support:

- Thousands of hotels
- High-volume bookings
- Real-time room availability
- Multi-region scaling